

GitHub 연동 · 업로드 없이 대화로 올린다

1 준비 (1회)

Claude Code에 "GitHub에 올리게 gh 로그인 도와줘" → 안내대로 브라우저 인증. 계정 연결은 한 번이면 끝

2 첫 업로드

"이 폴더를 GitHub 저장소로 만들어 올려줘" → 저장소 생성 · 커밋 · 푸시까지 Code가 수행

3 수정 반영

"변경사항 커밋하고 푸시해줘" 한 줄 · Vercel이 연결돼 있으면 자동 재배포까지 이어진다

4 안전 규칙

키 · 비밀번호 파일은 올리지 않기 · "키 파일은 제외되게 .gitignore 만들어줘"라고 먼저 부탁

웹 드래그 업로드는 언제나 유효한 플랜 B입니다. 연동은 편의이지 필수가 아닙니다.

Vercel 연동 · push가 곧 배포다

1 연결 (1회)

vercel.com → Add New → Project → GitHub 저장소 Import. 교육 때 배포한 그 연결 그대로

2 자동 배포

이후 Code가 푸시할 때마다 Vercel이 감지해 재배포 · 주소(URL)는 그대로 유지

3 CLI 방식 (선택)

"vercel CLI로 이 폴더 배포해줘" · 설치와 로그인 1회 후엔 명령 한 줄로 배포

4 확인

배포 URL 새로그침 · Vercel 대시보드 Deployments 탭에서 배포 이력 확인

수정 → 푸시 → 자동 배포. 교안 마지막에 그린 그 바퀴가 이렇게 굴러갑니다.

Supabase 연동 · MCP로 DB가 대화 상대가 된다

1 연결 (1회)

Code에 "Supabase MCP 연결해줘. 처음엔 읽기 전용으로" → 안내대로 로그인 · 승인 (명령 한 줄은 가이드 문서 참고)

2 조회

"water_quality에서 오염 의심 행 보여줘" · SQL을 몰라도 대화로 조회

3 관리

"테이블 스키마 확인해줘", "measure_date에 인덱스 만들어줘" · 대시보드 클릭이 대화가 된다

4 안전

처음엔 읽기 전용(read-only) 연결 권장 · Secret 키는 여전히 앱과 코드에 넣지 않는다

클릭으로 하던 모든 일이 대화가 됩니다. 단, 무엇을 어떤 단위로 저장할지는 여전히 사람이 정합니다.

워크플로우

1 PRD 수정

요구가 바뀌면 코드가 아니라 PRD부터 고친다 · 문제의 주소는 늘 PRD 안에 있다

2 구현

"PRD대로 고쳐줘" → Code가 수정 · step 파일이 곧 버전이라 언제든 되돌릴 수 있다

3 배포

"커밋하고 푸시해줘" → GitHub 반영 → Vercel 자동 재배포 · 같은 URL에서 새 버전

4 DB 점검

MCP로 "이번 주 저장된 건수 확인해줘" · 판정 기준이 바뀌면 PRD 3번과 테이블을 함께 갱신

교육에서 클릭으로 익힌 길을, 실무에서는 대화로 달립니다. 이 한 바퀴가 여러분 업무의 새 기본기입니다.

키 관리 · 매번 입력에서 무입력까지

1 1단계 · 입력창

교육 기본. 코드와 저장소에 비밀이 없다 · 시연과 학습에 최적, 단일 HTML의 정석

2 2단계 · 브라우저 기억

"키를 localStorage에 저장해 처음 한 번만 입력하게 해줘. 지우기 버튼도" · 개인 PC 실무용, 공용 PC 금지

3 3단계 · 서버 + 환경변수

"API 호출을 서버 함수로 옮기고 키는 환경변수에서 읽게 해줘" · 사용자가 키를 몰라도 되는 부서 공유용

4 철칙

.env를 깃에 올린 적이 있다면 지워도 히스토리에 남는다 · 키 재발급이 정답

입력창(교육) → 기억(개인) → 서버(부서). 단계마다 대화 한 줄로 올라갑니다.

현장 팁 I · 요청 한 줄이면 됩니다

1 휴대폰에서도

"휴대폰 화면에서도 보기 좋게 반응형으로 만들어줘"

2 보고서로 뽑기

"판정 결과를 A4 인쇄에 맞는 보고서 형식으로, 인쇄 버튼 추가" · 브라우저 인쇄에서 PDF 저장

3 데이터 백업

"테이블 전체를 CSV로 내려받는 백업 버튼을 만들어줘"

4 기간별 조회

"분기 선택 드롭다운을 추가해서 선택한 기간만 조회되게 해줘" · 분기가 쌓이면 필수

전부 오늘 만든 앱에 한 줄씩 없는 기능입니다. PRD 백로그에 적고 하나씩.

현장 팁 II · 커질 때와 막힐 때

1 느려질 때

"1,000건 넘으면 페이지를 나눠서 보여줘" + "자주 찾는 컬럼에 인덱스 만들어줘"

2 아무나 못 보게

"접속 코드를 입력해야 화면이 보이게 해줘" · 단, 가림막이지 보안이 아님. 민감 데이터는 공개 URL 금지

3 다른 파일 형식

"CSV 파일과 시트가 여러 개인 엑셀도 읽을 수 있게 해줘"

4 에러 만능 열쇠

F12 → Console의 빨간 메시지를 통째로 복사 → "이 에러 고쳐줘: (붙여넣기)"

기능 추가는 코드가 아니라 PRD 백로그에 먼저 적고, "백로그 1번을 구현해줘"로 시작합니다.

특수 과제 · 수집은 백로그로, 예측은 참고로

1

자동 수집 과제

크롤링·알림은 서버가 필요해 하루 범위를 넘는다 · 오늘은 "모아온 것을 넣으면 분류·요약"까지, 수집은 백로그(나중에 할 일 목록·PRD 7번)에

2

백로그 실현

"Vercel Cron으로 매일 아침 뉴스를 수집해 요약을 이메일로 보내줘" · 백로그가 곧 다음 스프린트 지시문

3

예측 과제

오늘은 집계 + 추세선 + 기준 초과 규칙 판정까지 · 예측 모델은 백로그, 실현할 땐 이동평균·추세 연장부터

4

예측의 원칙

추정값에는 "참고용 추정치" 표시 · 판정은 언제나 사람이 정한 기준(규칙)으로, AI 숫자로 행정 판단하지 않는다

범위를 좁히는 것은 포기가 아니라 예약입니다. 백로그가 다음 버전을 기다립니다.

비용과 사용량 · 사고 없이 오래 쓰기

1 지출 한도

워크스페이스에 Spend limit을 걸어두면 예산 사고가 원천 차단된다

2 만료일 습관

키에는 만료일 설정을 기본으로 · 폐기가 자동이 된다

3 사용량 확인

platform.claude.com 워크스페이스 화면에서 확인 · 429(요청 과다)는 1~2분 대기, 잦으면 서버 구조로 올릴 신호

4 Supabase 플랜

무료 플랜은 교육·시제품에 충분 · 부서 도구로 정착하면 용량과 백업 정책 확인

교육에서 클릭으로 익힌 길을, 실무에서는 대화로 달립니다. 이 아홉 장이 그 지도입니다.

워크스페이스와 내 키 만들기 · 콘솔 5분

1 콘솔 접속

platform.claude.com 로그인 → Settings. 계정이 없으면 가입 후 결제 수단 등록 (부서 협의)

2 워크스페이스 생성

Settings → Workspaces → 새로 만들기 (예: keco-수질) → 목록에서 wrkspc_로 시작하는 ID 확인

3 API 키 생성

Settings → API keys → Create key → 이름·만료일 입력 → 워크스페이스 지정(핵심) → sk-ant- 키는 한 번만 표시, 즉시 복사·보관

4 앱에 입력

워크스페이스를 지정한 키는 키만 넣으면 동작 · 지정 없이 만든 개인 키는 앱의 워크스페이스 ID 칸에 wrkspc_ ID를 함께 입력

만료일을 걸어두면 폐기가 자동입니다. 키는 비밀번호와 같습니다. 코드에 적지 말고, 앱 입력창으로만 쓰십시오.